



UNIVERSITÀ
DI TRENTO

SmartVase

An IoT Automated Greenhouse

Project for the course:
**Embedded Software for
the Internet of Things**

Presentation By:
**Giacomo Radin
Antonio Possemato
Francesco Vaona
Sebastian Fia**

Project Outline



UNIVERSITÀ
DI TRENTO

A plant in a normal vase cannot survive without continuous human care. To solve this we created **Smartvase**: an IoT vase that makes plants grow reliably and autonomously.

Features:

- Light detection, movement towards light, UV lighting
- Ultrasensors for Movement
- Humidity detection, automatic irrigation
- Periodic image capture and leaf health analysis on edge
- Mobile app to monitor and control the system

Main Hardware components

- Arduino Mega: direct hardware control of sensors and actuators
- ESP32 Standard(HUB): communication hub of the system
- ESP32-CAM: image capture and edge analysis

Project members contributions

- We mainly worked together in the fablab from the same pc
- At the end we divided the sections to finish the project





UNIVERSITÀ
DI TRENTO

Internal Communication and Control (HUB and MEGA)

Communication

- Hub and Mega communicate via serial/UART (Protobuf messages)
- Mega reads and controls all sensors
- Hub interfaces with outside world (more on this later)

Failure Handling

- Watchdog for both MCUs
- Heartbeat
- Custom serial CLI for debugging

Core Algorithms and Logic

```
// Selects the best sector (brightest or darkest) after a 360° scan
inline int8_t careBestScanSector(const float* meanAdc, uint8_t n, bool seekLight) {
    int8_t best = -1;
    float bestV = 0.0f;

    for (uint8_t i = 0; i < n; ++i) {
        const float v = meanAdc[i];

        // Ignore missing or invalid sensor readings (fail-safe)
        if (isnan(v) || v < 0.0f) continue;

        // Find maximum if seeking light, or minimum if seeking shade
        if (best < 0 || (seekLight ? (v > bestV) : (v < bestV))) {
            best = (int8_t)i;
            bestV = v;
        }
    }
    return best;
}
```



Memory leak risks on the Mega were solved by avoiding dynamic strings and using fixed buffers

External Communication

Components

- HiveMQ: MQTT broker
- Firebase: persistent database (telemetry, logs, images...)

Communication paths

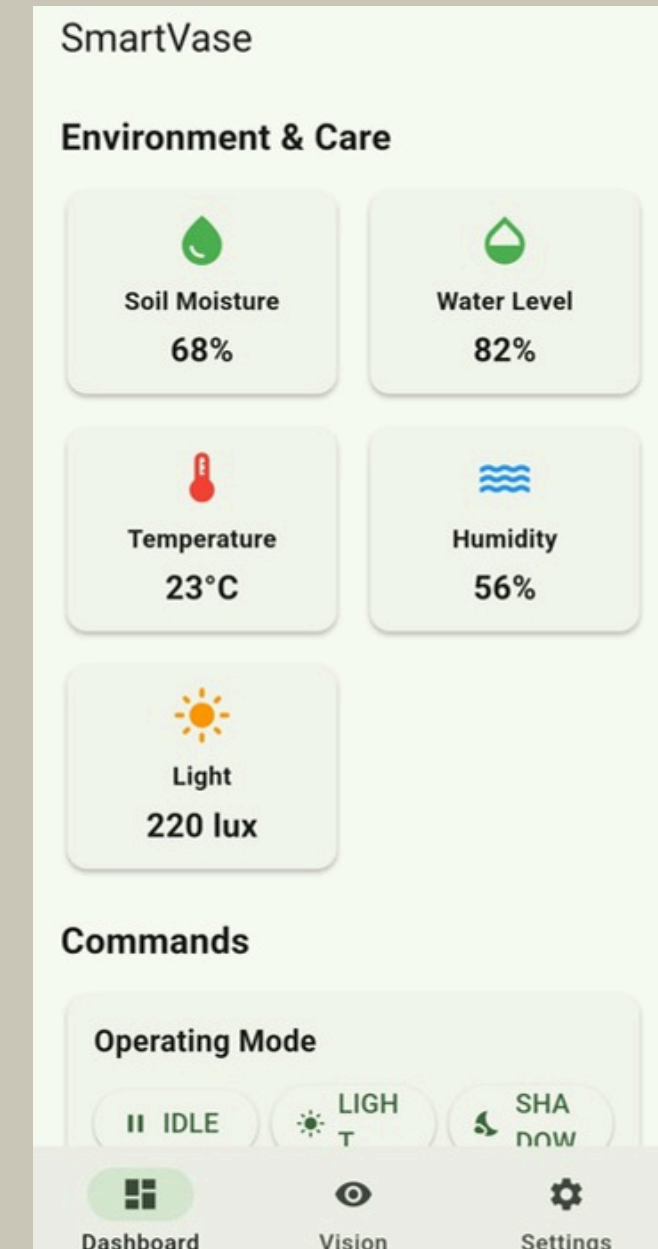
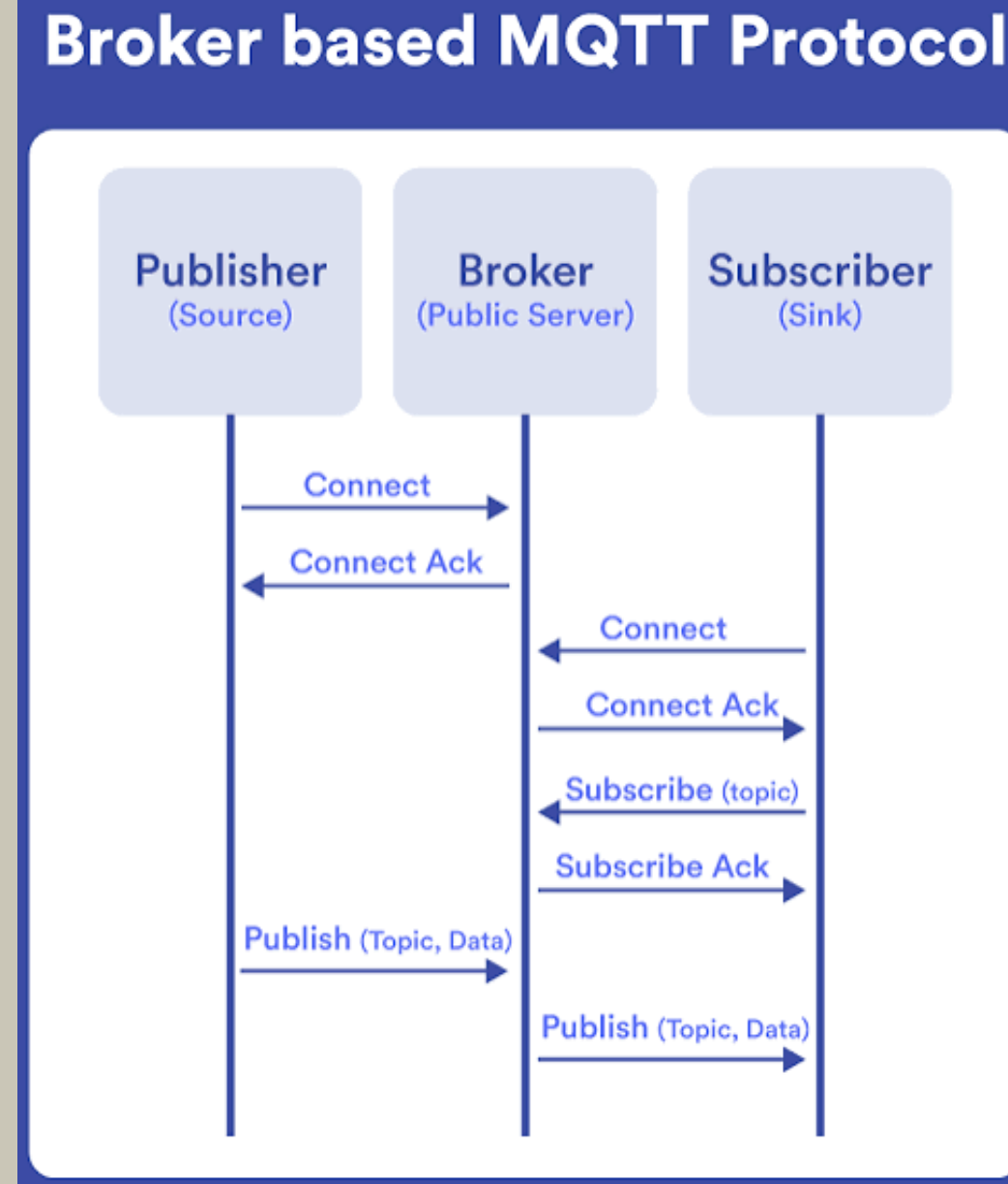
- Vase sends data and receives commands via MQTT (messages handled via FreeRTOS queues):
ESP32 HUB ↔ *HiveMQ* ↔ *Firestore* ↔ *Mobile App*
- Vase sends images and analyses (FirebaseClient.h)
ESP32 CAM → *Firestore/Storage* → *Mobile App*

Problems encountered

- Complexity (HUB)
- Communication latency (CAM, image upload)
- Crash on CAM if listening to Firebase (not enough RAM)

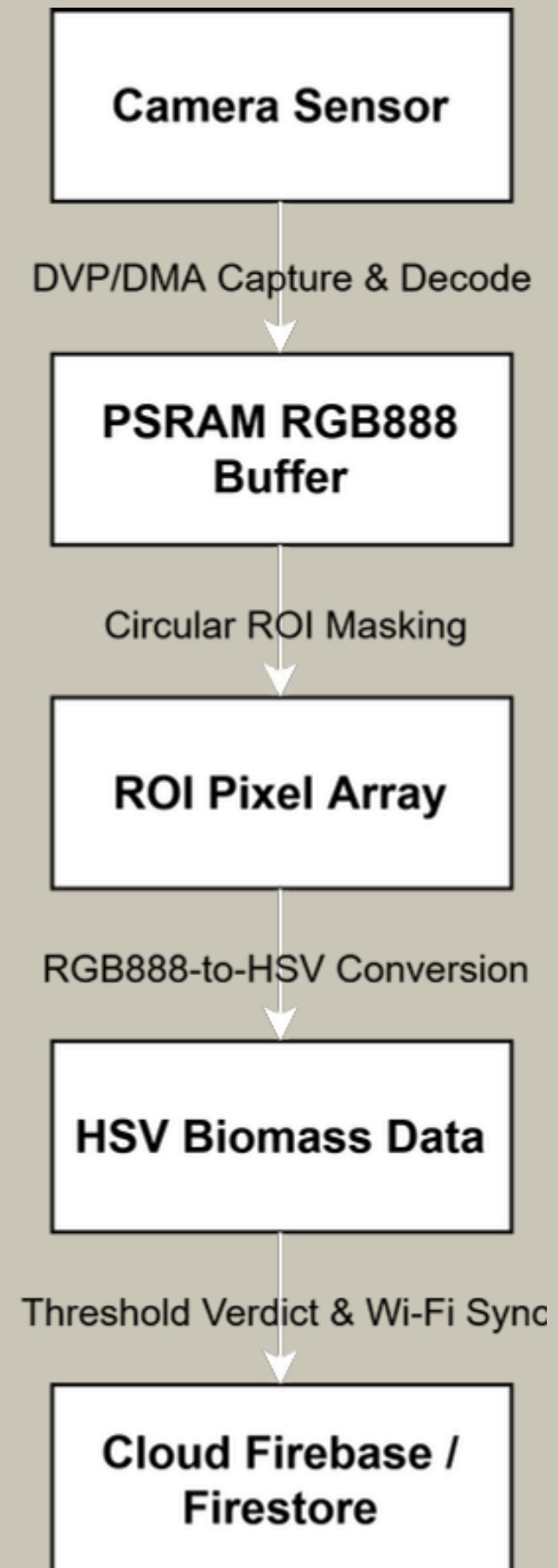
Solutions

- Complexity on the HUB → FreeRTOS
- Logs to Firestore to monitor state
- Latency and resources on CAM → preiodic image updates (no “request capture” on app)



On-Edge Vision & Diagnostics

- **On-Device Edge Vision:** Configures the **OV2640** sensor via **I2C** (SCCB) and captures video frames over an **8-bit DVP parallel bus using DMA**, decoding JPEG into PSRAM RGB888 pixel arrays without CPU blocking
- **Analysis & Diagnostics:** Applies dynamic circular **ROI masking** and **HSV color segmentation** in memory to decouple chromaticity from the picture brightness, then diagnosing plant health in real time
- **Cloud Telemetry:** Sends periodic **JPEG snapshots** to Firebase Storage and health diagnostics **JSON payloads** to Cloud Firestore via Wi-Fi



Future Work

- ML system for vision, create labeled dataset taking pictures with CAM
- Request image capture on CAM via MQTT
- Additional app features (e.g. control movement directly, images timelapse)
- Docking Station for battery recharge and water refill





UNIVERSITÀ
DI TRENTO

Thanks for your attention



Presentation By:
Giacomo Radin
Antonio Possemato
Vaona Francesco
Sebastian Fia